

Interação com APIs

10

Integração Full-Stack com Angular

Nesta aula, consolidamos toda a trilha de consumo de APIs integrando um front-end Angular a uma API real e completa — sem desenvolvimento de backend próprio — simulando um projeto corporativo com autenticação, CRUD, interceptors, guards e boas práticas de segurança.



A Situação de Aprendizagem

Contexto Corporativo

Uma **software house** recebeu a demanda de construir um **Painel Corporativo de Gestão de Publicações Internas**, onde colaboradores autenticados possam gerenciar postagens em tempo real. Toda integração deve ser feita consumindo uma API externa pronta — sem backend próprio.

Pergunta-Problema

Como construir uma aplicação Angular integrada a uma API completa, reproduzindo um fluxo real de mercado?

Esta é a questão central que guiará toda a aula. O aluno assume o papel de desenvolvedor front-end em um projeto real.

Por que a DummyJSON?

A **DummyJSON** foi escolhida como API oficial desta aula por reunir, em um único endereço, todos os recursos necessários para consolidar a trilha completa de consumo de APIs com Angular.



Autenticação

Login com usuário e senha, retornando token de acesso JWT.



CRUD Completo

GET, POST, PUT, PATCH e DELETE em recursos reais.



Busca e Filtros

Endpoint de pesquisa e paginação para listagens dinâmicas.



Documentação Clara

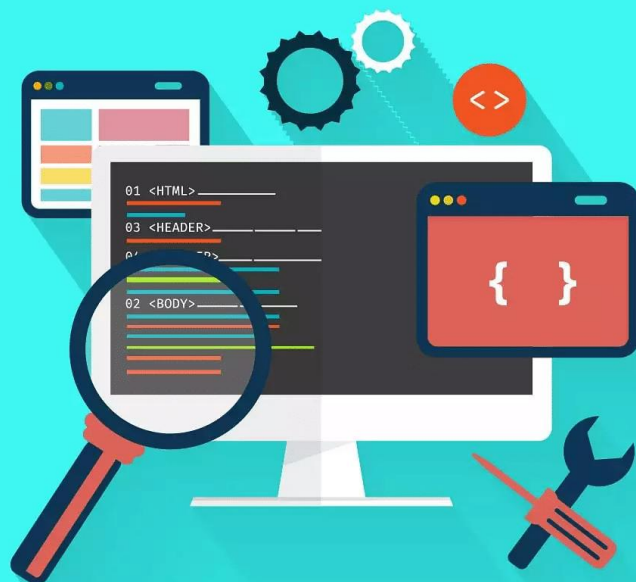
Payloads realistas e documentação completa para uso didático.



Uma única API permite consolidar toda a trilha. Usuários, produtos, posts, comentários e muito mais — tudo em dummyjson.com.

Revisão da Jornada

Antes de integrar, vale lembrar o caminho percorrido. Cada aula anterior contribuiu com uma peça do quebra-cabeça que agora se completa.



1

HTTP / HTTPS

Protocolo de comunicação e segurança na camada de transporte.

2

REST e JSON

Padrão arquitetural e formato de contrato entre sistemas.

3

GET / POST / PUT / DELETE

Métodos HTTP para operações completas de CRUD.

4

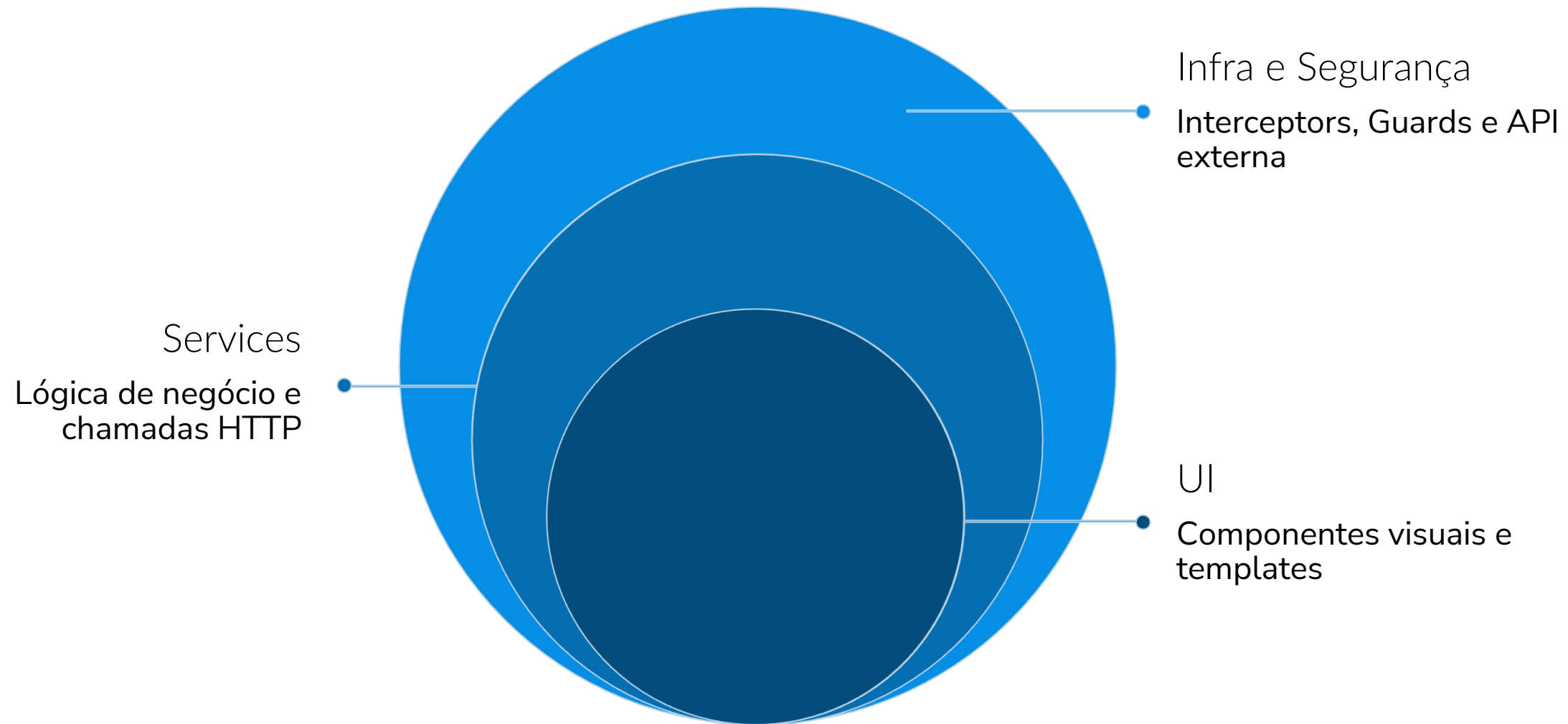
Segurança e Boas Práticas

Tratamento de erros, tokens, guards e interceptors.

☐ **Agora tudo converge em um projeto.** Esta aula é o ponto de chegada da trilha.

Arquitetura Angular Profissional

Uma aplicação Angular bem estruturada separa responsabilidades em camadas bem definidas. Cada camada tem um papel específico, facilitando manutenção, testes e escalabilidade — características essenciais em projetos corporativos reais.



Essa separação garante que cada responsabilidade fique isolada: componentes não conversam diretamente com a API — os serviços fazem essa ponte, enquanto interceptors e guards protegem toda a camada de comunicação.

Camadas da Arquitetura em Detalhe

UI Layer / Components

Responsável pela interface visual. Consome estado reativo exposto pelos services (Signals / Observables) e renderiza os templates HTML. Não deve conter lógica de negócio ou chamadas HTTP diretas.

Services

Centralizam toda a comunicação com a API. Usam o `HttpClient` do Angular para realizar requisições e retornar Observables para os componentes.

Interceptors

Interceptam todas as requisições HTTP automaticamente, injetando o token de autenticação no header `Authorization` antes de cada chamada.

Route Guards

Protegem rotas privadas. Verificam se o usuário está autenticado antes de permitir o acesso ao dashboard. Redirecionam para login em caso de sessão inválida.

Autenticação com Token

Endpoint Oficial

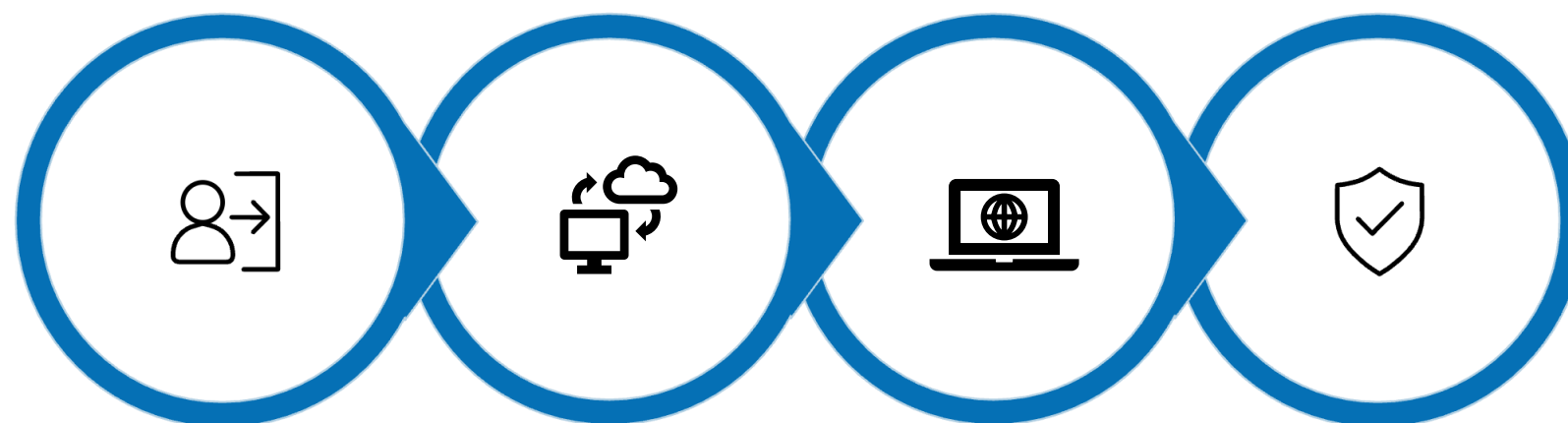
POST <https://dummyjson.com/auth/login>

Payload de Requisição

```
{  
  "username": "emilys",  
  "password": "emilyspass"  
}
```

Resposta da API

```
{  
  "id": 1,  
  "username": "emilys",  
  "email": "emily.johnson@x.dummyjson.com",  
  "accessToken": "jwt...",  
  "refreshToken": "jwt..."  
}
```



Login

JWT

Sessão

Rota

Auth Service – Implementação

O AuthService centraliza a autenticação da aplicação. O método login() envia username e password para a API e retorna um Observable<AuthResponse> contendo accessToken, refreshToken e dados do usuário autenticado.

```
private http = inject(HttpClient);  
private api = 'https://dummyjson.com/auth/login';  
login(credentials: Login) {  
  return this.http.post<AuthResponse>(this.api,  
    credentials);  
}
```

- ❗ O accessToken retornado é armazenado em sessionStorage e enviado automaticamente pelo HTTP Interceptor em todas as requisições protegidas via: Authorization: Bearer <token>

CRUD Completo com DummyJSON

A DummyJSON oferece endpoints RESTful completos para o recurso `/posts`, permitindo implementar todas as operações de CRUD de forma simples e padronizada.

Operação	Método	Endpoint
Listar posts	GET	<code>https://dummyjson.com/posts</code>
Criar post	POST	<code>https://dummyjson.com/posts/add</code>
Atualizar post	PUT	<code>https://dummyjson.com/posts/1</code>
Excluir post	DELETE	<code>https://dummyjson.com/posts/1</code>

✔ **CRUD completo usando a mesma API.** Todos os endpoints seguem o mesmo padrão REST, tornando a integração previsível e de fácil manutenção.

Post Service — Implementação Completa

O PostService centraliza todo o estado da aplicação: listagem, busca, paginação e CRUD. Utilizando Signals, o service atua como store local, mantendo a interface reativa e sincronizada com a API.

```
private http = inject(HttpClient);

posts = signal<Post[]>([]);
loading = signal(false);

carregar() {
  this.http
    .get<any>(endpoint)
    .subscribe(res => {
      this.posts.set(res.posts);
    });
}

criar(payload: Post) {
  this.http.post<Post>(
    `${this.api}/add`,
    payload
  );
}
```

- Signal Store local
- Busca dinâmica
- Paginação
- CRUD reativo
- Feedback visual (toast)

Interceptor HTTP — Injetando o Token

O **HTTP Interceptor** é um dos recursos mais poderosos do Angular para integração com APIs protegidas. Ele intercepta automaticamente *todas* as requisições HTTP da aplicação antes de enviá-las, permitindo adicionar o token de autenticação no header sem repetir código em cada serviço.

O que o Interceptor faz

- Lê o token salvo em `sessionStorage`
- Injeta o header `Authorization: Bearer <token>`
- Centraliza a autenticação em um único ponto
- Elimina repetição de código nos services

Benefícios Arquiteturais

- Princípio DRY (Don't Repeat Yourself)
- Manutenção simplificada
- Fácil substituição de estratégia de auth
- Possibilidade de adicionar lógica de retry e refresh token

Interceptor HTTP — Injetando o Token

```
export const authInterceptor:
HttpInterceptorFn = (req, Next) => {

  const token = authService.getToken();

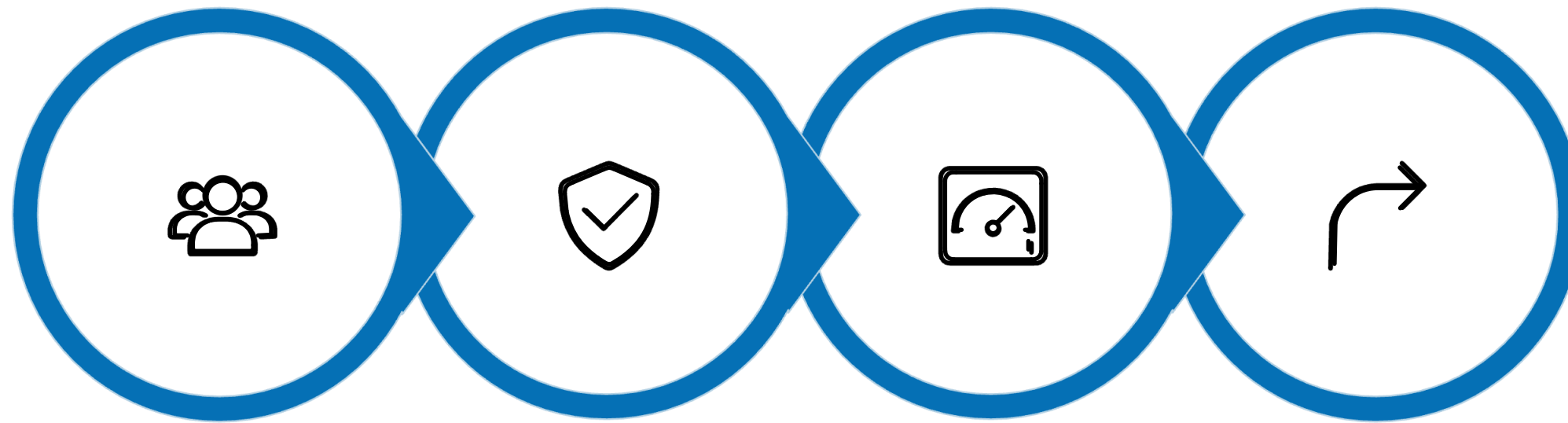
  if (!token) {
    return next(req);
  }

  const authReq = req.clone({
    headers: {
      Authorization:
        `Bearer ${token}`
    }
  });

  return next(authReq);
};
```

Route Guard — Protegendo o Dashboard

O **AuthGuard** protege as rotas privadas da aplicação. Antes de renderizar qualquer componente protegido, o guard verifica se o usuário possui um token válido em sessão. Caso contrário, redireciona imediatamente para a tela de login.



Solicita Acesso

Verifica Token

Acesso
Liberado

Redireciona
Login

`canActivateFn()`
Guard funcional do Angular 21.

`Router.navigate(['/login'])`
Redireciona quando não autenticado.

`AuthService.isAuthenticated()`
Verifica token salvo em sessionStorage.

Route Guard — Protegendo o Dashboard

```
export const authGuard:
CanActivateFn = () => {

  const authService =
    inject(AuthService);

  const router =
    inject(Router);

  if (
    authService.isAuthenticated()
  ) {
    return true;
  }

  router.navigate(['/login']);
  return false;
};
```


Fluxo Completo: Login → Dashboard



Este fluxo representa o padrão de autenticação utilizado em aplicações corporativas reais. O guard age como porteiro, garantindo que apenas usuários autenticados acessem áreas privadas, enquanto o interceptor garante que todas as chamadas à API carreguem as credenciais corretas.

Busca e Filtros Dinâmicos

Endpoint de Pesquisa

GET `https://dummyjson.com/posts/search?q=love`

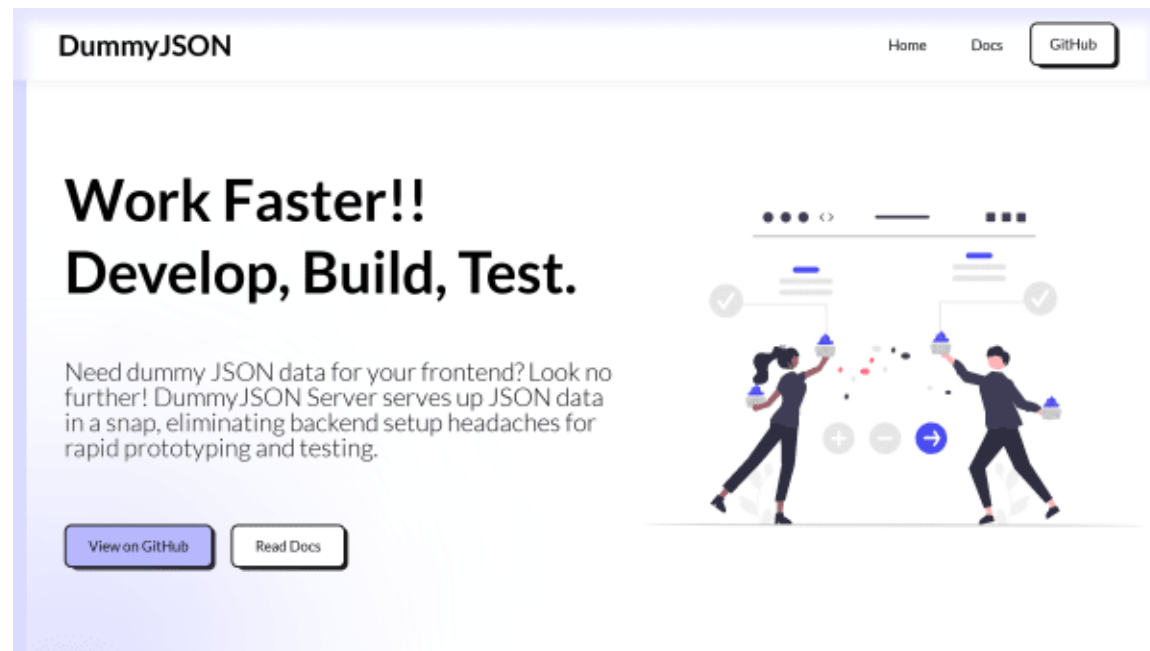
A DummyJSON oferece um endpoint dedicado para busca textual nos posts. O parâmetro `q` aceita qualquer string e retorna os posts correspondentes, permitindo implementar uma busca em tempo real com excelente experiência de usuário.

Funcionalidades disponíveis

- Busca por palavra-chave no conteúdo
- Paginação com `limit` e `skip`
- Filtro por categorias e tags
- Ordenação por campos específicos

Aplicação na UX

- Campo de busca reativo no dashboard
- Listagem atualizada dinamicamente
- Indicador de loading durante a busca
- Mensagem de "nenhum resultado" com fallback



Paginação com DummyJSON

A API suporta paginação nativa através dos parâmetros `limit` e `skip`, essenciais para lidar com grandes volumes de dados sem sobrecarregar a interface.

```
// Exemplo: buscar 10 posts a partir do 20º registro  
GET https://dummyjson.com/posts?limit=10&skip=20
```

```
// Resposta inclui metadados de paginação:  
{  
  "posts": [...],  
  "total": 150,  
  "skip": 20,  
  "limit": 10  
}
```

 Utilize os campos `total`, `skip` e `limit` retornados pela API para calcular o número de páginas e controlar a navegação no componente de paginação do dashboard.

Sessão Expirada e Fallback

Dois cenários críticos que toda aplicação corporativa deve tratar adequadamente:

Sessão Expirada (401)

Quando o token expira, a API retorna status `401 Unauthorized`. O interceptor deve capturar esse status e:

- Limpar o token do `sessionStorage`
- Redirecionar o usuário para `/login`
- Exibir mensagem: "Sessão expirada, faça login novamente"

Fallback Visual

Quando a API está indisponível ou retorna erro inesperado, a aplicação deve exibir um estado de fallback:

- Mensagem amigável ao usuário
- Botão de "Tentar novamente"
- Log do erro no console para debugging
- Estado vazio ilustrado (empty state)



O Projeto

Corporate Feed Manager

Uma aplicação Angular completa que simula um painel corporativo de gestão de publicações internas, integrando todos os conceitos da trilha em um único projeto funcional e profissional.

Funcionalidades do Corporate Feed Manager

1

Login e Logout

Autenticação com token JWT via DummyJSON. Sessão persistida no sessionStorage e encerrada com limpeza completa.

2

Dashboard Protegido

Rota privada protegida por AuthGuard. Acesso liberado apenas para usuários com token válido.

3

Listagem e Busca

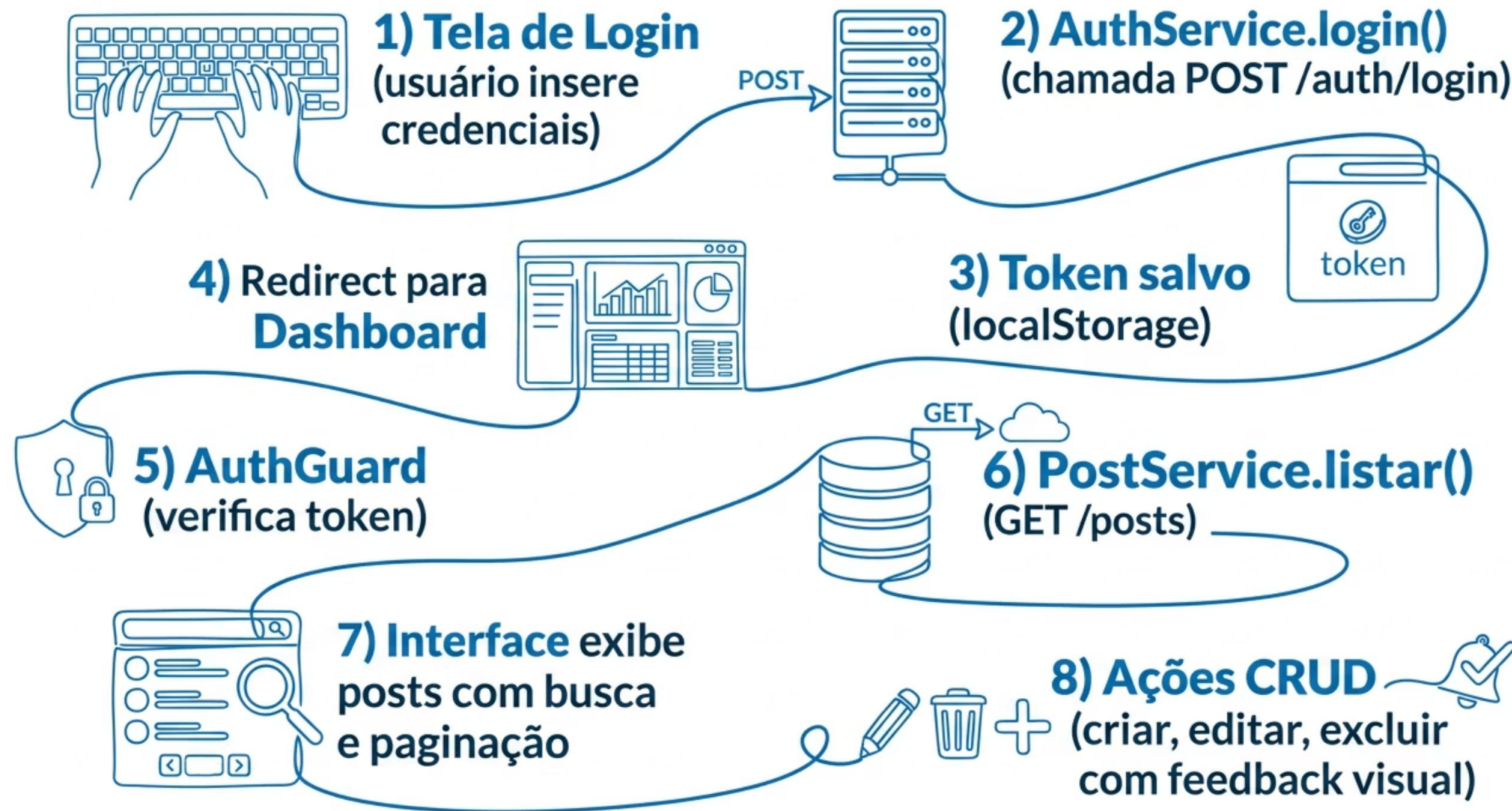
Exibição paginada de posts com campo de busca reativa e filtros dinâmicos.

4

CRUD Completo

Criação, edição e exclusão de postagens com feedback visual (toast) em cada operação.

Fluxo Completo da Aplicação



Este fluxo representa a jornada completa do usuário na aplicação — do login ao gerenciamento de conteúdo — demonstrando como cada camada arquitetural contribui para uma experiência corporativa fluida e segura.

Implementando o Login Component

O componente de login utiliza Reactive Forms para captura das credenciais e Signals para gerenciamento de estado visual (loading e mensagens de erro). Após autenticação, o token é salvo e o usuário é redirecionado ao dashboard.

Template HTML

```
<form
  [formGroup]="form"
  (ngSubmit)="entrar()"
>
  <input
    formControlName="username"
    type="text"
  />

  <input
    formControlName="password"
    type="password"
  />

  <button type="submit">
    Entrar
  </button>
</form>
```

Component TypeScript

```
loading = signal(false);
erro = signal('');

entrar() {
  this.authService.login(
    this.form.getRawValue() as Login
  )
  .subscribe({
    next: (res) => {
      this.authService.salvarToken(
        res.accessToken
      );
      this.router.navigate([
        '/dashboard'
      ]);
    }
  });
}
```


Implementando o Post List Component

O componente de listagem renderiza os posts armazenados no PostService. Toda lógica de carregamento, busca, paginação e CRUD permanece centralizada no service, mantendo o componente simples e reativo.

```
@Component({
  selector: 'app-post-list'
})
export class PostList {

  postService =
    inject(PostService);

  constructor() {
    this.postService.carregar();
  }

  editar(post: Post) {
    this.postService.editar(
      post.id!,
      { body: 'novo texto' }
    );
  }

  excluir(id?: number) {
    if (!id) return;

    this.postService.remover(id);
  }
}
```

```
@if (postService.loading()) {
  <p>Carregando...</p>
}

@else {
  @for (
    post of postService.posts();
    track post.id
  ) {
    ...
  }
}
```

- inject()
- Signals
- @if
- @for
- CRUD reativo
- estado centralizado

Boas Práticas de Segurança Front-End

- Nunca expor o token na URL
Sempre enviar o token no header `Authorization`, nunca como query parameter visível.
- Validar token antes de cada requisição
O interceptor deve verificar se o token existe e não está expirado antes de injetá-lo.
- Limpar sessão no logout
Remover token do `sessionStorage` e redirecionar para login ao encerrar sessão.
- Proteger todas as rotas privadas
Aplicar `AuthGuard` em todas as rotas do dashboard — nunca confiar apenas no front-end para controle de acesso.



Reflexão Técnica

Antes de considerar o projeto concluído, o bom desenvolvedor avalia criticamente a solução entregue. Reflita sobre as seguintes questões com sua equipe:

Arquitetura Escalável?

Os módulos, serviços e componentes estão bem organizados?
A aplicação suportaria novos recursos sem refatoração profunda?

Token Seguro?

O token está sendo armazenado e transmitido de forma segura? A sessão expirada é tratada corretamente pelo interceptor?

Services Bem Organizados?

Cada service tem uma única responsabilidade? Os componentes estão delegando a lógica corretamente aos services?

Pronto para o Mercado?

A aplicação trata erros, exibe feedback visual, lida com loading states e possui tratamento de sessão expirada?

Como Evoluir o Projeto

O **Corporate Feed Manager** é uma base sólida. Veja como evoluir a aplicação para o próximo nível profissional:



Refresh Token

Implementar renovação automática do token antes da expiração, mantendo a sessão ativa sem interrupção.



Lazy Loading

Carregar módulos sob demanda para melhorar a performance inicial da aplicação.



Testes Unitários

Adicionar testes para services, interceptors e guards usando Viteste + Angular Testing Library — padrão corporativo.

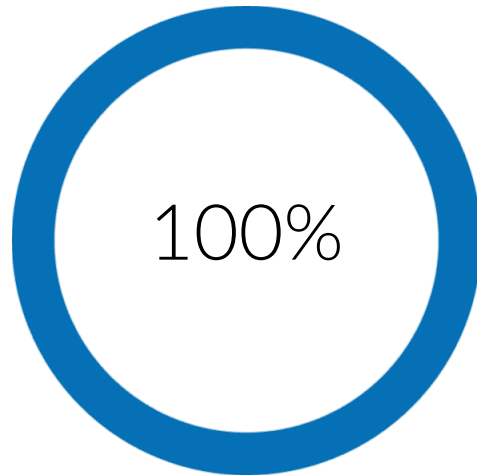


Gerenciamento de Estado

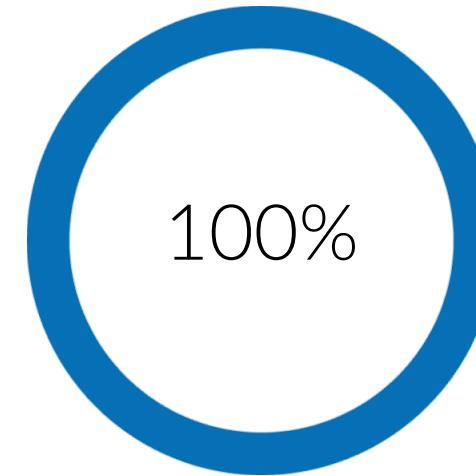
Integrar NgRx ou Signals para gerenciar estado global de forma previsível e escalável.

Evidências de Competência

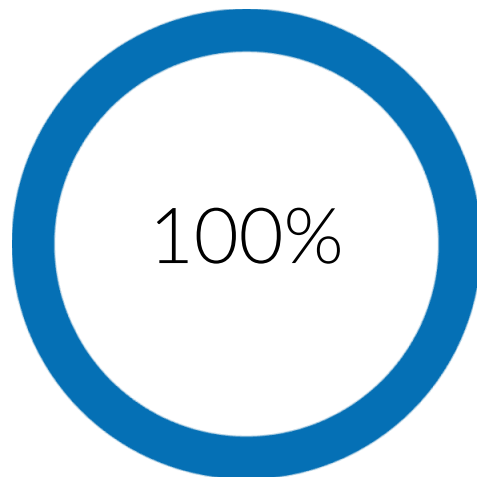
Ao concluir esta aula e o projeto integrador, o aluno demonstra ter desenvolvido competências técnicas alinhadas ao perfil profissional do mercado de desenvolvimento de sistemas.



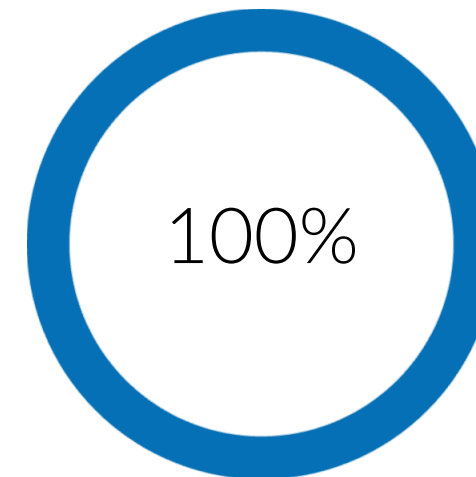
Autenticação Real
Autenticar em API real com token JWT e gerenciar sessão.



CRUD Completo
Consumir endpoints protegidos com GET, POST, PATCH e DELETE.



Interceptor + Guard
Implementar interceptor de token e guard de rota privada.



Integração Profissional
Estruturar arquitetura front-end com separação de responsabilidades.

Checklist Final do Projeto

Utilize este checklist para validar se o **Corporate Feed Manager** está completo e pronto para entrega:

- ✓ Login com usuário e senha via DummyJSON
- ✓ Token salvo no sessionStorage após autenticação
- ✓ AuthGuard protegendo a rota /dashboard
- ✓ Functional Interceptor injetando Bearer Token automaticamente
- ✓ Listagem reativa de posts com Signals + paginação
- ✓ Busca dinâmica integrada ao endpoint /posts/search?q=
- ✓ Criação de novo post via POST /posts/add
- ✓ Edição de post via PATCH /posts/:id
- ✓ Exclusão de post via DELETE /posts/:id
- ✓ Loading state reativo com Signals
- ✓ Toast reativo para sucesso e erro nas operações CRUD
- ✓ Logout com limpeza de sessão e redirecionamento

Conexão com o Mundo do Trabalho

As competências desenvolvidas nesta aula refletem diretamente o dia a dia de um **Desenvolvedor Front-End Júnior** em uma software house ou equipe de produto. Empresas buscam profissionais que saibam:

Integrar APIs

Consumir serviços externos com segurança e eficiência.

Organizar Código

Arquitetura limpa, separação de responsabilidades, reusabilidade.

Tratar Falhas

Aplicações robustas que respondem bem a erros e indisponibilidades.

Segurança Básica

Proteger rotas, gerenciar tokens e não expor dados sensíveis.



